ECE-006 Wi-Fi Hotspot Voucher & Session Accounting Engine

Design Document — Team E010

Voucher management: Greeshma Reddy R | Session management: Gowthami J | Python 3.10+, standard library only

1. Overview

The engine is command-file driven (no UI). It reads **commands.txt**, loads voucher types and issued vouchers from CSV, processes a day of LOGIN / USAGE / LOGOUT events in timestamp order, and writes four files: **session_log.csv**, **voucher_report.csv**, **unused_vouchers.csv** and **errors.csv**.

Area	Owner	Files
Voucher management: types, issuing, activation, 24 h validity, balances, charging, voucher and unused reports	Greeshma	models.py, voucher_manager.py, tests/test_vouchers.py
Session management: LOGIN/LOGOUT/USAGE, device limit, idle timeout, TIME exhaustion, timestamp ordering, session log, rejection reasons	Gowthami	session_engine.py, tests/test_sessions.py
Command-file driver: reads commands.txt, calls both parts, writes the four CSVs	Both	main.py

2. Data model

Class	Fields	Notes
VoucherType	prefix, voucher_type (TIME|DATA), quantity (minutes or MB), price, max_devices, type_id	type_id (VT-01 ...) is printed in the report
Voucher	code, voucher_type, issued_at, remaining_balance, activated, activation_time, expiry_time, active_devices, devices_used	Balance starts at the type's quantity (float, so running time can be fractional). devices_used = every MAC that ever logged in
VoucherManager	voucher_types, types_by_id, vouchers, counters	Loads CSVs, looks vouchers up, produces the reports

3. Business-rule register

Every rule in the brief has an ID. *Owner* says which part of the code enforces it.

ID	Rule	Owner
BR-01	Activation: the 24 h window starts at the voucher's first successful LOGIN, not at issue. The window is [activation, activation + 24 h). A LOGIN at or after the end is rejected EXPIRED, including exactly 24 h.	Voucher
BR-02	Time vouchers: minutes consumed LOGIN to LOGOUT per device, summed across devices. A session that would exceed the balance is cut at the exact second the balance reaches zero. Balance never goes below 0 (charge_minutes).	Voucher + Session
BR-03	Data vouchers: MB from USAGE events are deducted. When remaining MB reaches zero, all sessions on that voucher end at that event's timestamp. Balance never goes below 0 (charge_mb).	Voucher + Session
BR-04	Concurrency: at most max_devices devices logged in at once; an extra LOGIN is rejected DEVICE_LIMIT.	Session
BR-05	Idle timeout: 10 minutes with no USAGE logs the device out at last activity + 10 min; time vouchers stop consuming from then.	Session
BR-06	Rounding: a time session is charged in whole minutes rounded up (math.ceil of its connected minutes). Data rounding is a switch, see D1: raw MB as received (default) or each event rounded up to the next MB.	Voucher + Session
BR-07	Report: revenue counts a voucher on its day of activation; vouchers never activated are listed separately in unused_vouchers.csv.	Voucher
BR-08	Events are processed by timestamp, then LOGIN < USAGE < LOGOUT, then event_id, so out-of-order rows are handled.	Session
BR-09	Idle logouts and cut-offs due at or before an event's timestamp are applied before that event.	Session

ID	Rule	Owner
BR-10	A LOGIN from a MAC already connected on that voucher is ignored: no new session, no log row, no error line (sample README).	Session
BR-11	LOGIN check order: unknown voucher, duplicate MAC, expired, balance exhausted, device limit.	Session + Voucher
BR-12	USAGE/LOGOUT with no active session (never logged in, idle-logged-out, or cut off) is rejected NOT_LOGGED_IN and nothing is deducted (see D4).	Session
BR-13	Malformed input rows are reported (INVALID_EVENT) and never crash the run; bad voucher types are rejected with a message.	Both

4. How the two parts work together

main.py reads commands.txt. The session engine never edits balances itself; it calls the voucher module.

Call	Purpose	Rule
VoucherManager.load_types(path)	Read voucher_types.csv, validate and register each type	BR-13
VoucherManager.load_vouchers(path)	Read vouchers.csv; keep the given codes and issue times; skip unknown types and duplicate codes	BR-13
VoucherManager.get(code)	Return the voucher or None (None means UNKNOWN_VOUCHER)	BR-11
VoucherManager.is_expired(code, ts)	True when ts >= activation + 24 h; never for an unactivated voucher	BR-01
VoucherManager.activate(code, ts)	First successful LOGIN sets activation_time; later calls never move it	BR-01
Voucher.charge_minutes(n)	Deduct n (possibly fractional) minutes, capped at the balance	BR-02
Voucher.charge_mb(mb, round_up)	Deduct MB, capped at the balance; round_up selects policy D1	BR-03, BR-06
Voucher.devices_used	Set of MACs that ever logged in; its size is the devices_used column	BR-07
VoucherManager.report_rows(date, now)	Rows for vouchers activated on that date, sorted by voucher_code; status judged at `now`	BR-07
VoucherManager.unused_rows()	Vouchers never activated, in issue order	BR-07
SessionEngine.process(events)	Sort events (BR-08), advance the clock before each one (BR-09), handle LOGIN/USAGE/LOGOUT, snapshot the report at midnight	BR-02 to BR-12

4a. Idle timeout and cut-off in timestamp order (viva)

Events are sorted once. Before each event at time *t*, **advance_to(t)** repeatedly finds the earliest pending engine-made closure that is due at or before *t*: a session's last activity + 10 minutes, or the moment a shared TIME balance reaches zero (balance / number of active devices minutes from now). It moves the clock there, deducts time from every active TIME voucher, closes what is due (IDLE_LOGOUT or CUT_OFF), and repeats. Only then is the event itself handled. So a USAGE at 11:20 for a device that went idle at 11:16 finds no session and is rejected NOT_LOGGED_IN, and the IDLE_LOGOUT row (11:16) is already in the log. After each calendar day the clock moves to midnight and the report snapshot for that day is taken.

5. Output formats owned by the voucher module

voucher_report.csv — one row per voucher activated on a REPORT date, ordered by voucher_code:
`report_date,voucher_code,type_id,kind,activated_at,consumed,remaining,devices_used,revenue_rs,status`

revenue_rs is the type price. consumed and remaining are plain numbers (minutes for TIME, MB for DATA). status is EXHAUSTED if the balance is 0, else EXPIRED if the end of the report date is >= activation + 24 h, else ACTIVE (D3).

unused_vouchers.csv:
`voucher_code,type_id,issued_at`

6. Test plan (22 tests)

Test (file)	Checks	Rule
test_load_types_and_vouchers (vouchers)	4 types and 60 vouchers load; all unused before any login	BR-07
test_activation_starts_24h_window, test_24h_boundary (vouchers)	Activation never moves; 23:59:59 valid, exactly 24 h expired	BR-01

Test (file)	Checks	Rule
test_data_rounds_up_and_stops_at_zero, test_time_never_negative (vouchers)	Charging caps at the balance; round_up=True turns 24.6 into 25	BR-02, BR-03
test_report_revenue_on_activation_day, test_exhausted_and_expired_status (vouchers)	Voucher appears on its activation date only; EXHAUSTED beats EXPIRED	BR-07
test_bad_type_input_does_not_crash (vouchers)	Text quantity, zero devices, duplicate prefix give clean errors	BR-13
test_two_devices_exhaust_together (sessions)	Both cut at 10:30:00, 30 min each; later LOGOUTs rejected	BR-02, BR-12
test_single_device_cut_before_logout (sessions)	Cut at 17:00:00; 17:15 LOGOUT rejected	BR-02
test_idle_logout_then_late_usage... (sessions)	IDLE_LOGOUT 11:16; USAGE 11:20 rejected and not deducted	BR-05, BR-12
test_idle_logout_on_time_voucher... (sessions)	Charged 19 minutes	BR-05, BR-06
test_device_limit_and_repeated_mac_ignored (sessions)	Third device DEVICE_LIMIT; repeated MAC ignored	BR-04, BR-10
test_login_exactly_24h... (sessions)	06:29:59 accepted, 06:30:00 EXPIRED	BR-01
test_unknown_voucher_and_logout_without_login (sessions)	UNKNOWN_VOUCHER and NOT_LOGGED_IN	BR-12
test_data_raw_policy..., test_data_round_up_policy... (sessions)	0.4 MB left then 1 MB: charge 0.4, all sessions end 12:20; round-up ends 12:16	BR-03, D1
test_out_of_order_rows..., test_malformed_event... (sessions)	Sorting, same-second priority, INVALID_EVENT	BR-08, BR-13
FullSampleRun (3 tests)	Sample pack: 60 vouchers, 7 unused, 692 log rows, 13 errors; planted scenarios; log in time order	all

Run: `python -m unittest discover -s tests -v` — all 22 tests pass.

7. Edge cases from the brief

Edge case	Handling
LOGIN exactly 24 h after activation	Rejected EXPIRED (window is half-open, BR-01)
Two devices on a 60-minute voucher, both logged in 30 minutes	Balance is exhausted together at the 30-minute mark; both sessions cut at that second (BR-02)
USAGE arriving after the device was idle-logged-out	Rejected NOT_LOGGED_IN; no deduction (BR-12)
LOGOUT for a device that never logged in	Error line NOT_LOGGED_IN (BR-12)
Data voucher with 0.4 MB remaining receives a 1 MB event	Raw policy (D1): charge only the 0.4 MB left, balance 0, all sessions end at that timestamp. Round-up policy: the voucher is already exhausted one event earlier (12:16).

8. Decisions made where the brief is silent

#	Decision	Alternative (switch)
D1	Data MB is deducted as received (24.6 MB charges 24.6). The sample README accepts either policy if it is stated; the 0.4 MB edge case in the brief only arises with raw deduction.	Round each event up to the next MB (BR-06 wording): DATA_ROUND_UP = True in session_engine.py. One sample voucher would end at 12:16 instead of 12:20.
D2	A repeated LOGIN of a connected MAC is ignored: no new session, no log row, no error (sample README).	Log it as REJECTED / DUPLICATE_LOGIN.
D3	Report status is judged at the end of the report date (midnight snapshot).	Judge at the last processed event.
D4	A USAGE event after the device was cut off or idle-logged-out is rejected NOT_LOGGED_IN, consistent with the README's LOGOUT/USAGE examples.	BALANCE_EXHAUSTED: USAGE_AFTER_CUTOFF_REASON in session_engine.py. Affects 5 rows of the sample.
D5	Sessions are not ended when the 24 h window closes mid-session; only new logins are rejected.	Cut running sessions at expiry.

#	Decision	Alternative (switch)
D6	On a TIME voucher the remaining column shows the live balance at the event time; the clock runs continuously, a session row charges its minutes rounded up.	Show only closed-session balance.
D7	Known limit: if timestamps carried seconds, balance would be consumed continuously while the row charge is rounded up. All sample timestamps are whole minutes, so both agree.	Round the balance per session.